

university of groningen

Performance Engineering: Lecture #04

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-10

Overview



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Content



1. **Benchmarking**
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Benchmarks: Introduction



- **Benchmark** = Program which is executed to measure the performance of a system or a system component
- **Benchmarks categorization**
  - **Work-load type**
    - Real application work-load
    - Micro-benchmarks: Programs which stimulate a particular feature of the system
  - **Code type**
    - Kernels: Relevant pieces of real application programs
    - Synthetic benchmarks: Fake programs which try to match the profile and behaviour of real programs
- **Motivations** for using benchmarks
  - Comparison of different architectures
  - **Create knowledge for performance engineering**
    - What are the hardware characteristics of the target system?
    - What is the highest attainable performance?

Arithmetic Operations Throughput



- **High-Performance Linpack Benchmark (HPL)**
  - Solve a dense $N \times N$ system of linear equations

$$A \cdot x = b$$

  - Basis for Top500 ranking of supercomputers [Top500]
  - Top500 **benchmarking rules**
    - Problem size is not fixed
    - Algorithm is prescribed (LU factorisation with partial pivoting, double precision)
    - Code is not fixed, reference implementation provided: `https://netlib.org/benchmark/hpl/`
- Several benchmarks that aim at maximising filling of the floating-point pipelines
  - Mainly suitable for exploring hardware characteristics
  - Example: uarch-bench, UBENCH-x86

Memory Performance



- **STREAM**
  - De facto standard benchmark for the measurement of **CPU memory bandwidth**
  - Includes several benchmarks: `copy`, `scale`, `add`, `triad`
  - `https://github.com/jeffhammond/STREAM`
- **BabelStream** [Deakin et al.; 2018]
  - Support of various programming models (including GPU programming models)
  - Augmented STREAM by a `dot` benchmark
- **tinymembench**
  - Benchmark for memory bandwidth and **latency**

Network Performance



- **Intel MPI Benchmarks**
  - Benchmarks various communication patterns implemented in MPI
  - `https://github.com/intel/mpi-benchmarks.git`
- **NetGauge**
  - Network parameter measurement tool that supports benchmarking of different network protocols and communication patterns
  - `https://htor.inf.ethz.ch/research/netgauge/`
- **perftest**
  - Collection of tests for micro-benchmarking network technologies supporting RDMA
  - `https://github.com/linux-rdma/perftest`

Application-Based Benchmarks



- **NAS Parallel Benchmark Suite (NPB)**
  - Set of programs designed to help evaluate the performance of parallel computers
  - Implemented kernels and pseudo-applications:
    - IS: Integer Sort, random memory access
    - EP: Embarrassingly Parallel
    - CG: Conjugate Gradient, irregular memory access and communication
    - MG: Multi-Grid on a sequence of meshes, long- and short-distance communication, memory intensive
    - FT: discrete 3D fast Fourier Transform, all-to-all communication
    - BT: Block Tri-diagonal solver
    - SP: Scalar Penta-diagonal solver
    - LU: Lower-Upper Gauss-Seidel solver
  - Source:
    - Original version (Fortran): `https://www.nas.nasa.gov/software/npb.html`
    - Reimplementation in C: `https://github.com/benchmark-subsetting/NPB3.0-omp-C`

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Introduction to Computational Dwarfs



- **Defining dwarfs** = An approach to capture patterns of computation and communication common to a class of important applications
  - Approach introduced 2006 [Asanovic et al.; 2006]
  - Various dwarfs already identified in earlier work
- **Goal**: Identify commonalities between numerical applications to exploit common knowledge and experience about
  - Performance characteristics (e.g. arithmetic intensity)
  - Best-practices for implementation on a given architecture

Computational Dwarfs: Overview



Dwarfs for HPC

1. Dense linear algebra
2. Sparse linear algebra
3. Spectral methods
4. N-body methods
5. Structured grids
6. Unstructured grids
7. MapReduce (Monte Carlo)

Other Dwarfs

8. Combinational logic
9. Graph traversal
10. Dynamic programming
11. Back-track and branch + bound
12. Graphical models
13. Finite state machines

Computational Dwarf: Dense Linear Algebra



- Key data objects are **densely populated matrices or vectors**
- Typical **memory access pattern**:
  Unit-stride or fixed-stride regular sequential access
- Example: $y_i \leftarrow \sum_j A_{ij} \cdot x_j$

$$\begin{pmatrix} 1 & 4 & 5 & 6 \\ 0 & 3 & 2 & 6 \\ 1 & 2 & 3 & 1 \\ 5 & 4 & 3 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 48 \\ 36 \\ 18 \\ 30 \end{pmatrix}$$

- Popular library with dense linear algebra kernels: **BLAS (Basic Linear Algebra Subprograms)**
  - Different implementations available:
    - Open source: NETLIB BLAS, OpenBLAS, BLIS
    - Closed source: MKL (Intel), ESSL (IBM), Arm Performance Library (Arm)
  - Classification of operations
    - BLAS-1: scalar, vector and vector-vector
    - BLAS-2: matrix-vector
    - BLAS-3: matrix-matrix

Computational Dwarf: Sparse Linear Algebra



- Key data objects are **matrices or vectors with many zeros**
- To avoid storing matrices with many zeros, **compressed formats** are used
  - Popular format: **Compressed Sparse Row (CSR)**
    - Number of non-zero entries in matrix is denoted as N_{nz}
    - Vector A_i ($i = 0, \ldots, N_{\mathrm{nz}} - 1$) stores the values of the matrix (collected row-by-row)
    - Vector C_i ($i = 0, \ldots, N_{\mathrm{nz}} - 1$) stores column index of each element of A
    - Vector R_i ($i = 0, \ldots, N_{\mathrm{nz}}$) stores the number of non-zero elements up to and not including row i with $R_0 = 0$
  - Example
    - $A_i = \{4, 7, 6, 9\}$
    - $C_i = \{3, 0, 1, 0\}$
    - $R_i = \{0, 1, 3, 3, 4\}$

$$\begin{pmatrix} 0 & 0 & 0 & 4 \\ 7 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 9 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 16 \\ 19 \\ 0 \\ 36 \end{pmatrix}$$

Computational Dwarf: Structured Grids (1/3)



- Applications perform periodic updates of **regular, multi-dimensional grids**
- Memory access features
  - Regular strided memory access
  - High spatial data locality, i.e. consecutive access to data that is close in memory
- Application areas
  - Heat transfer
  - Lattice Quantum Chromodynamics
  - Computational fluid dynamics using the Lattice Boltzmann Method

Computational Dwarf: Structured Grids (2/3)



Example: 2-dimensional Poisson equation

- Formulation in the continuum

$$-\frac{\partial^2 v(x,y)}{\partial x^2} - \frac{\partial^2 v(x,y)}{\partial y^2} = f(x,y)$$

- Discretisation of 2nd-order derivative

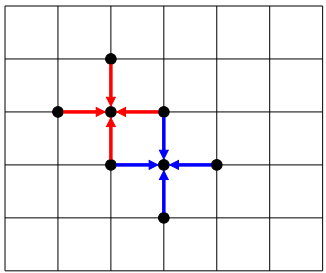
$$-\frac{\partial^2 v(x,y)}{\partial x^2} \leftarrow \frac{2v_{i,j} - v_{i-1,j} - v_{i+1,j}}{h^2}$$

- Discrete Poisson equaiton in 2 dimensions:

$$Tv = h^2 f \quad \text{where} \quad T = \begin{pmatrix} 4 & -1 & 0 & \cdots \\ -1 & 4 & -1 & \cdots \\ \vdots & \vdots & \ddots & \end{pmatrix}$$

Computational Dwarf: Structured Grids (3/3)

Graphical representation of Tv:



Observations:

- Matrix T acts as a stencil operator
- Any element of vector v is reused 4 times ☞ data locality

Computational Dwarf: Spectral Methods

- **Discrete Fourier Transform (DFT)**

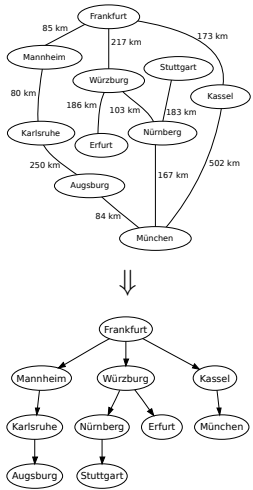
$$y_k = \sum_{n=0}^{N-1} x_n \, e^{-\frac{2\pi i}{N} n \, k}$$

- DFT using the radix-2 Cooley-Tukey algorithm

$$y_k = \sum_{m=0}^{N/2-1} x_{2m} \, e^{-\frac{2\pi i}{N} (2m) \, k} + \sum_{m=0}^{N/2-1} x_{2m+1} \, e^{-\frac{2\pi i}{N} (2m+1) \, k}$$

- Approach can be generalised to $N = N_1 \cdot N_2$:
  - Perform N_1 DFTs of size N_2 (original data layout)
  - Multiply by complex roots of unity + transform of $N_1 \times N_2$ matrix
  - Perform N_2 DFTs of size N_1 (new data layout)

Computational Dwarfs: Graph Traversal



[Wikimedia Commons]

- Applications that **traverse a number of objects** and **examine characteristics of those objects** such as would be used for search
- **Example problem**: Create search tree for city connections
- **Popular algorithm**: **Breadth-First Search (BFS)**
  - BFS explores all of the neighbour nodes at the present depth prior to moving on to the nodes at the next depth level
- **Typical characteristics**
  - Indirect table lookups
  - Little computation

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Model



- **Model** = Material or virtual artefacts that map to an origin [Thalheim; 2014]
  - An **origin** can be a material world or an artificial artefact
  - Model and origin only share relevant features (truncation property)
  - Models are designed for a specific purpose (purpose property)
  - Models intended to generate specific values and benefit (added value property)
- **Motivation for using models**
  - Performance modelling
  - Functional modelling and specification
  - Validation and verification
  - ...
- **Types of models**
  - Mathematical models
  - Event simulation
  - Signal level simulations (in soft- or hardware)
  - ...

Performance, Bandwidth, and Throughput



- **Performance** = Amount of work W per time unit

$$b(W) = \frac{W}{\Delta t(W)}$$

  - While this definition does not specify work, it needs to be well-defined, e.g. execution of a specific **workload** (also called problem size)

- **Bandwidth** = Amount of data passing a **particular interface** per time unit
  - It is important to define this interface

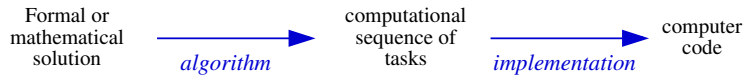
- **Throughput** = Amount of work items processed per time unit
  - Example: Number of floating-point operations per time unit

$$B_{\mathrm{fp}} = \frac{N_{\mathrm{fp}}}{\Delta t}$$

Performance Model



- **Performance model** = Estimate of latency Δt as a function of different parameters
- **Parameters**
  - Problem to be solved
  - Chosen algorithm
  - Implementation
  - Computer architecture
  - Problem size



Motivation for Performance Modelling

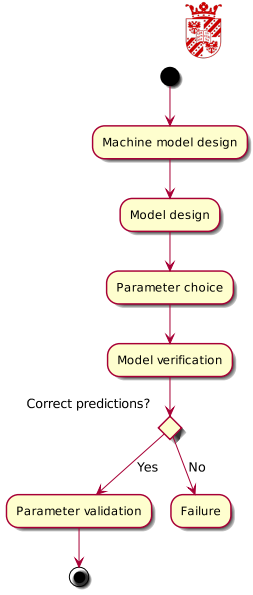


[Hoefler et al., 2011]

- **Modelling for performance optimization**
  - Identify optimization opportunities
  - Choose the right algorithm
- **Modelling during software development**
  - E.g., software design following the waterfall model including the steps: analysis, design, implementation, testing, maintenance
- **Modelling during system design and proposal**
  - Create basis for trade-off decisions
- **Modelling during deployment and testing**
  - Compare expected and obtained performance
- **Modelling during operation**
  - Compare expected and operational performance (e.g., to identify performance degradation)

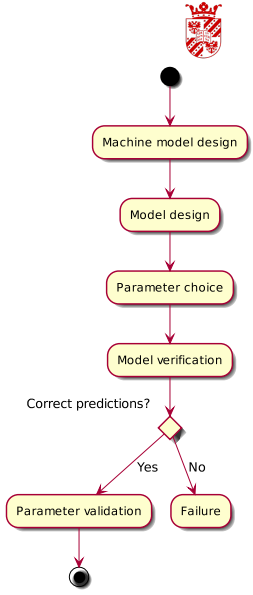
Modelling Workflow (1/2)

- **Machine model design**
  - Any performance model is based on a simplified model of the hardware (and/or software) architecture
- **Performance Model design**
  - Analytic model
  - Semi-analytical model
  - Simulation model
- **Model parameter choice/determination**, e.g.
  - Determine parameters using data sheets
  - Fit parameters to performance data
  - Apply training/learning methods



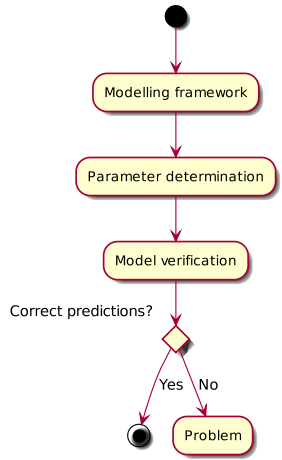
Modelling Workflow (2/2)

- **Model verification**
  - Use model to make predictions
  - Compare predictions to measurements
- Optional: **Parameter validation**
  - Do parameters have expected size?



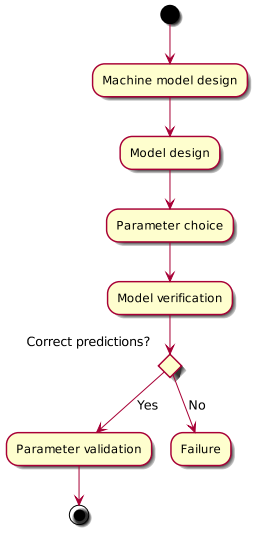
Black Box Modelling

- **Black box modelling** approaches
  - Statistical methods to parametrize performance data
  - Machine learning techniques
- Black box modelling methods may be independent from architecture and application
  - Rely on pattern matching and curve fitting
- **Advantage(s)**
  - Simple to apply
- **Disadvantage(s)**:
  - Does not create understanding
    - Failure of the validation step merely showcase limitation of the approach



White Box Modelling

- **White box modelling** approaches
  - Choice/design of both, machine model and performance model based on knowledge and/or assumptions
  - Example: Knowledge about the memory hierarchy
- In case of **failures**
  - Select new or adjust the machine model
  - Select new or adjust the performance model
- **Disadvantage(s)**:
  - Much more difficult
- **Advantage(s)**:
  - Creation of new insight
    - Model assumptions are challenged



(White Box) Performance Model Classification



- **Analytic performance models**
  - Performance is expressed with purely analytic expressions
  - Parameters are fixed from data-sheet information
- **Semi-analytical performance models**
  - Performance is expressed with purely analytic expressions
  - Parameters are determined through fits to performance data
- **Simulation models**
  - Simulates behaviour of machine model for given input stimuli
  - May include cycle-accurate simulation of digital logic

Modelling Different Quantities



- **Which quantities to model?**
  - **Latency Δt**
    - Time-to-solution in case of latency of full application
  - **Energy-to-solution ΔE**
  - **Resource utilization**
    - Example: Number of cache misses
  - **Workload requirements**
    - Example: Number of floating-point operations
- **Classification of quantities involved in modelling**
  - **Deterministic quantities**
    - Number of floating-point operations
  - **Statistical quantities**
    - Latency Δt

Workload



- **Workload** = A definition of a particular work requests imposed on the underlying computing resources
- The definition may involve
  - Software application
  - Command-line parameters
  - Input data
  - ...
- Workload documentation is important as **performance is typically a function of the work**

Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. Final

Proxy Architectures (1/2)



[Ang et al. (Sandia/LBNL), 2014]

- **Proxy architecture** = Parameterized version of a machine model
  - Parameter set limited to describe hardware capability and capacity of relevant components
  - **Example parameters**
    - Number of cores
    - Memory bandwidth, floating-point throughput
    - Capacity of caches
- **Why parametrized model?**
  - Model different versions of the same types of components, e.g. CPU
  - Explore architectural design space

Proxy Architectures (2/2)



[Ang et al. (Sandia/LBNL), 2014]

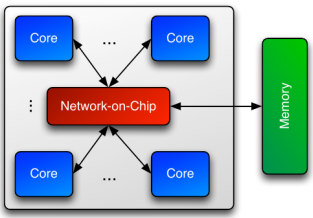
- **Different scopes**
  - Processor level
  - Node level
  - System level
- **Different level of details**
  - As little details as possible, as many details as necessary

Proxy-Architectures for CPUs (1/2)

[Ang et al. (Sandia/LBNL), 2014]

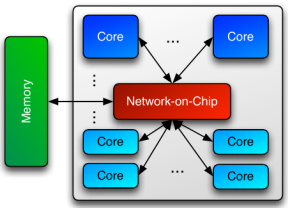
Homogeneous multi-core processor:

- Multiple cores of same type
- Processor-internal network
- External memory



Heterogeneous multi-core processor:

- Multiple cores of different type
- Processor-internal network
- External memory

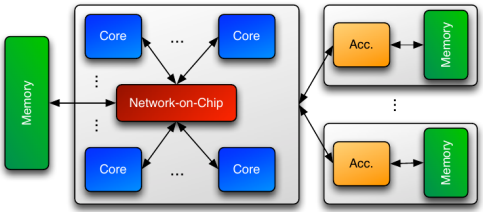


Proxy-Architectures for CPUs (2/2)

[Ang et al. (Sandia/LBNL), 2014]

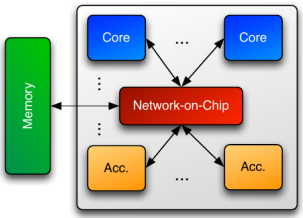
Multi-core processor plus discrete accelerator:

- Example: CPU + GPU

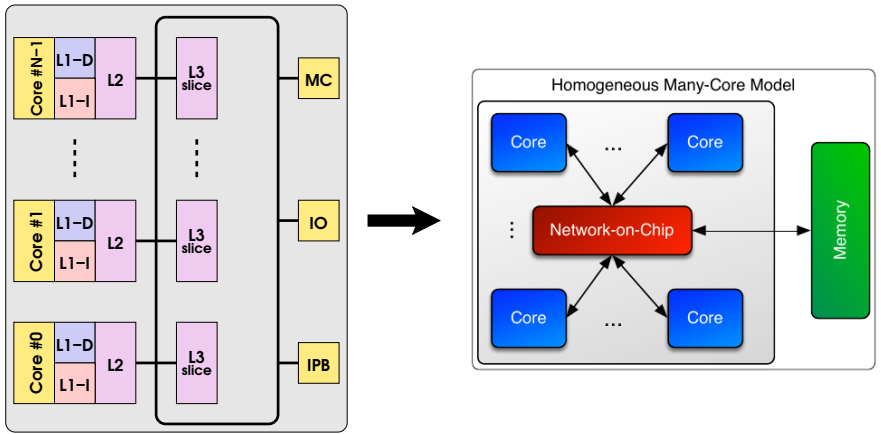


Multi-core processor plus integrated accelerators:

- Example: AMD APU



Proxy-Architecture for Intel Xeon E5 (Haswell)



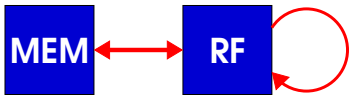
Abstract Machine Model (1/3)



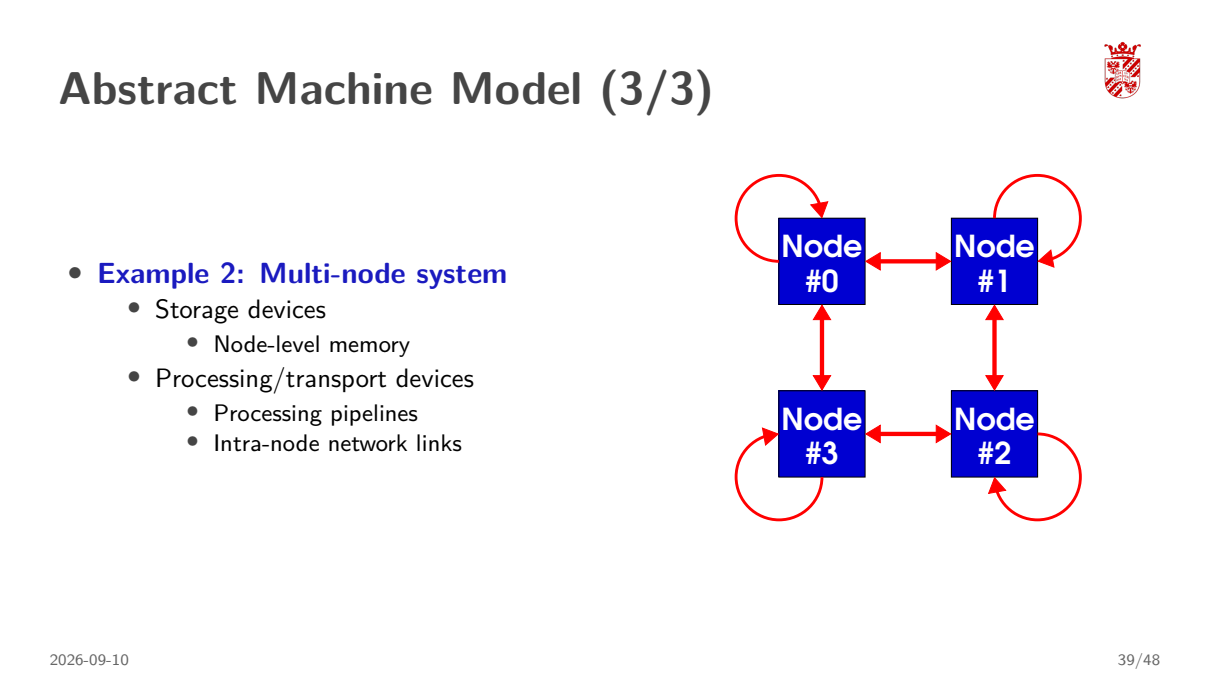
- **Machine** = set of interconnected devices
  - **Storage devices**
  - **Processing/transport devices**
- **Storage devices**
  - Examples
    - External memory
    - Register file
  - Parameter: capacity
- **Processing/transport devices**
  - Examples
    - Arithmetic pipeline
    - Memory bus
  - Possible parameters
    - Bandwidth/throughput
    - Start-up latency

Abstract Machine Model (2/3)

- **Graphical representation**
  - Vertices = storage devices
  - Edges = Processing/transport devices
- **Example 1: Processor architecture**
  - Storage devices
    - External memory (MEM)
    - Register file (RF)
  - Processing/transport devices
    - Memory bus
    - Processing pipeline



Abstract Machine Model (3/3)



- **Example 2: Multi-node system**
  - Storage devices
    - Node-level memory
  - Processing/transport devices
    - Processing pipelines
    - Intra-node network links

Content

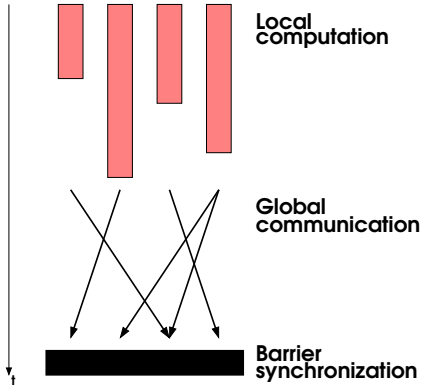


1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. **BSP Model**
6. Final

Bulk-Synchronous Parallel (BSP) Model (1/2)

[Valiant, 1990]

- **Machine model**:
  - Parallel machine comprising set of processors
  - Each processor has local memory
  - Interconnect network allows to communicate packets of fixed size
- **Performance modelling approach**:
  - Computation is divided in **supersteps**
    - Operations on local data
    - Send/receive packets
    - Barrier synchronisation



Bulk-Synchronous Parallel (BSP) Model (2/2)



- **Performance model parameters**:
  - P ... Number of processors
  - g ... Gap = time required to communicate one packet
  - L ... "Latency" = minimum duration of a superstep
  - S ... Number of supersteps
- **Latency** of superstep i and total latency

$$\Delta t_i = w_i + g\,h_i + L, \quad \Delta t = \sum_i \Delta t_i = W + g\,H + L\,S$$

  where $W = \sum_i w_i$ and $H = \sum_i h_i$
- **Superstep work depth** w_i is the largest amount of work performed in superstep i
- h_i **is largest number of packets sent or received** by any processor in superstep i

Cannon Algorithm (1/2)



- **Computational task**

$$C_{i,j} \leftarrow \sum_{k=0}^{N-1} A_{i,k} B_{k,j} \quad (i,j = 0, \ldots, N-1)$$

- **Parallelisation strategy**
  - Organise P processing units in a 2-dimensional mesh: $P = p^2$
  - Perform 2-dimensional decomposition of matrices A, B, C
    - Each processing unit holds a block of size $(N/p) \times (N/p)$
  - All elements of C stay fixed on a processing unit
    - All computations are performed on the processing unit that will hold the result
  - Blocks of matrices A and B need to be communicated
    - Canon algorithm consists of a description how to shift the blocks

Cannon Algorithm (2/2)

Shifting of blocks according to Cannon:

- Initial alignment step
- Blocks of A are shifted left
- Blocks of B are shifted upwards

Example $N = 3$, $P = 3^2$:

Initial state	Initial alignment	Shift #1	Shift #2
$\begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{pmatrix}$	$\begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,1} & A_{1,2} & A_{1,0} \\ A_{2,2} & A_{2,0} & A_{2,1} \end{pmatrix}$	$\begin{pmatrix} A_{0,1} & A_{0,2} & A_{0,0} \\ A_{1,2} & A_{1,0} & A_{1,1} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{pmatrix}$	$\begin{pmatrix} A_{0,2} & A_{0,0} & A_{0,1} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} & A_{2,0} \end{pmatrix}$
$\begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} \\ B_{1,0} & B_{1,1} & B_{1,2} \\ B_{2,0} & B_{2,1} & B_{2,2} \end{pmatrix}$	$\begin{pmatrix} B_{0,0} & B_{1,1} & B_{2,2} \\ B_{1,0} & B_{2,1} & B_{0,2} \\ B_{2,0} & B_{0,1} & B_{1,2} \end{pmatrix}$	$\begin{pmatrix} B_{1,0} & B_{2,1} & B_{0,2} \\ B_{2,0} & B_{0,1} & B_{1,2} \\ B_{0,0} & B_{1,1} & B_{2,2} \end{pmatrix}$	$\begin{pmatrix} B_{2,0} & B_{0,1} & B_{1,2} \\ B_{1,0} & B_{1,1} & B_{2,2} \\ B_{0,0} & B_{2,1} & B_{0,2} \end{pmatrix}$

BSP Model for Cannon Algorithm

- Number of superstep $S = \sqrt{P}$
- Local computation at each superstep

$$w_i \propto \left(\frac{N}{\sqrt{P}}\right)^3$$

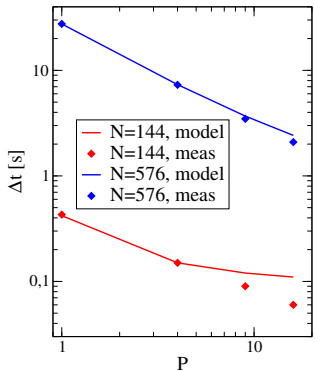
- Number of packets sent at each superstep

$$h_i = 2\left(\frac{N}{\sqrt{P}}\right)^2$$

- Overall latency:

$$\Delta t = \gamma \frac{N^3}{P} + g\,2\,\frac{N^2}{\sqrt{P}} + L\sqrt{P}$$

[Goudrea et al., 1999]



Content



1. Benchmarking
2. Computational Dwarfs
3. Introduction to Modelling
4. Machine Models
5. BSP Model
6. **Final**

Summary Lecture #04



- Benchmarking
  - Motivation for benchmarking
  - Benchmarks categorization
  - Introduced various benchmarks
- Computational dwarfs
  - Dense and sparse linear algebra, structured grids, spectral methods, graph traversal
- Introduction to modelling
- Machine models versus performance models
- Modelling techniques
  - BSP model

Questions?



[Wiki Media]